Documento de Arquitetura

Emile Vargas Bordin
Felipe Souza Flores
Natan de Aragão Müller

PUCRS — Escola Politécnica
Fundamentos de Processamento Paralelo e Distribuído
Prof. Marcelo Veiga Neves

13 de maio de 2026

1 Introdução

Este documento apresenta a arquitetura do projeto desenvolvido para a disciplina de Fundamentos de Processamento Paralelo e Distribuído.

O projeto consiste na implementação concorrente do jogo "Taco Gato Cabra Queijo Pizza", utilizando a linguagem Go. Durante o desenvolvimento, foram utilizados conceitos como goroutines, channels, `select` e shutdown gracioso.

A arquitetura foi organizada de forma concorrente, permitindo execução simultânea dos jogadores, renderização em tempo real e controle central do jogo.

2 Repositório do Projeto

O código-fonte do projeto está disponível no **GitHub do Projeto**.

3 Arquitetura Geral

A aplicação foi dividida em módulos responsáveis por diferentes partes do sistema.

- **deck.go**: criação, embaralhamento e distribuição das cartas;
- **player.go**: definição dos jogadores e comportamento concorrente dos bots;
- **game.go**: controle da lógica e estado global da partida;
- **main.go**: inicialização das goroutines, criação dos channels e coordenação geral da aplicação.

A comunicação entre os componentes ocorre exclusivamente através de channels.

4 Diagrama de Goroutines e Channels

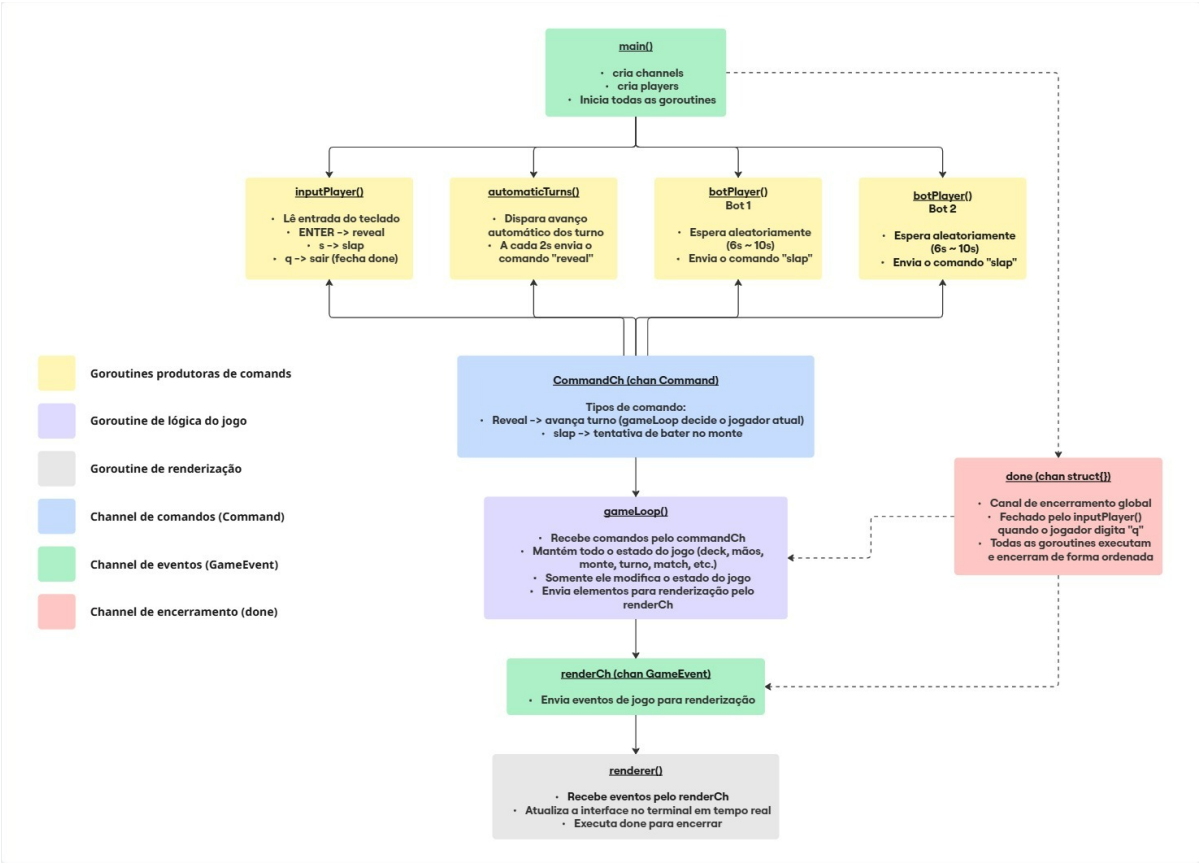

Figura 1: Diagrama de goroutines e channels

A Figura 1 apresenta as goroutines utilizadas, os channels responsáveis pela comunicação e o fluxo de dados da aplicação.

- jogadores e bots enviam comandos para o channel `commandCh`;
- a goroutine `gameLoop` processa os comandos e controla o estado do jogo;
- eventos de renderização são enviados para o `renderCh`;
- a goroutine `renderer` atualiza as informações no terminal;
- a goroutine `automaticTurns` envia comandos automáticos para avanço da partida;
- o encerramento ocorre através do channel `done`.

5 Papel das Goroutines

Cada goroutine possui uma responsabilidade específica dentro da aplicação.

- **inputPlayer**: recebe comandos do jogador humano;
- **botPlayer**: executa os jogadores automáticos em paralelo;
- **gameLoop**: controla o estado global e a lógica da partida;
- **renderer**: realiza a renderização em tempo real;
- **automaticTurns**: envia comandos automáticos de revelação de cartas.

6 Decisões de Projeto

O controle do estado da aplicação foi centralizado na goroutine `gameLoop`. Jogadores e bots não alteram diretamente as estruturas compartilhadas, enviando comandos através de channels.

Essa abordagem reduziu problemas de concorrência e facilitou a sincronização da aplicação.

Além disso, os channels foram utilizados como principal mecanismo de comunicação entre as goroutines.

7 Análise de Concorrência

O projeto foi desenvolvido buscando evitar problemas como deadlocks e goroutine leaks.

A centralização do estado na goroutine `gameLoop` reduziu acessos simultâneos às estruturas compartilhadas.

O encerramento da aplicação ocorre através do channel `done`, permitindo shutdown gracioso das goroutines.

8 Uso de Select

A estrutura `select` foi utilizada para tratar múltiplos eventos concorrentes.

Na goroutine `gameLoop`, o `select` recebe comandos e identifica sinais de encerramento através do channel `done`.

Nos bots, o `select` permite execução contínua enquanto o jogo permanece ativo e interrompe a goroutine quando o sistema é encerrado.

9 Conclusão

O projeto permitiu aplicar conceitos de programação concorrente utilizando a linguagem Go.

Durante o desenvolvimento, foram utilizados recursos como goroutines, channels, `select` e mecanismos de sincronização.

A arquitetura baseada em troca de mensagens contribuiu para manter o sistema organizado e reduzir problemas relacionados ao acesso concorrente aos dados compartilhados.

Referências

[1] PUCRS. **Materiais da disciplina de Fundamentos de Processamento Paralelo e Distribuído**. Prof. Marcelo Veiga Neves, 2026.

[2] GO AUTHORS. **The Go Programming Language Specification**. Disponível em: `https://go.dev/ref/spec`. Acesso em: 10 maio 2026.